

TP4

September 4, 2025

1 Temas Tratados en el Trabajo Práctico 4

- Representación del Conocimiento y Razonamiento Lógico.
- Estrategias de resolución de hipótesis: Encadenamiento hacia Adelante, Encadenamiento hacia Atrás y Resolución por Contradicción.
- Representación basada en circuitos.

1.1 Ejercicios Teóricos

1. ¿Qué es una inferencia?

La inferencia es el proceso mediante el cual un agente o sistema deduce nueva información a partir de hechos o conocimientos previos. En Inteligencia Artificial, la inferencia permite razonar: por ejemplo, a partir de reglas y datos conocidos, obtener conclusiones o decidir acciones.

Ejemplo lógico: “Si detecto que la oficina está sucia y mi regla dice que si está sucia debo limpiarla, entonces infiero que debo limpiar”.

Ejemplo probabilístico: “Si un sensor indica que hay un 80% de probabilidad de fallo en un motor dado cierto ruido, el sistema infiere que lo más probable es que el motor necesite mantenimiento”.

2. ¿Cómo se verifica que un modelo se infiere de la base de conocimientos?

Es una técnica lógica para saber si una sentencia (por ejemplo, una afirmación como “no hay un pozo en [1,2]”) se puede inferir lógicamente de una Base de Conocimientos (BC). - Modelos posibles: Se enumeran todos los mundos posibles (modelos) donde la BC es verdadera. - Evaluación: Se verifica si la sentencia también es verdadera en todos esos modelos. - Conclusión: Si es verdadera en todos los modelos donde la BC lo es, entonces se deriva de la BC (es una consecuencia lógica).

3. Observe la siguiente base de conocimiento:

$R1 : b \wedge c \rightarrow a$

$R2 : d \wedge e \rightarrow b$

$R3 : g \wedge e \rightarrow b$

$R4 : e \rightarrow c$

$R5 : d$

$R6 : e$

$R7 : a \wedge g \rightarrow f$

3.1 ¿Cómo se puede probar que $a = \text{True}$ a través del encadenamiento hacia adelante? Este método

3.2 ¿Cómo se puede probar que $a = \text{True}$ a través del encadenamiento hacia atrás? Este método

3.3 Exprese la base de conocimiento en su Forma Normal Conjuntiva. A continuación, demuestre

3.1 Por encadenamiento hacia adelante, de los hechos iniciales d y e se deduce b (R2) y c (R4)

3.2 Para demostrar que $a = \text{True}$ utilizando encadenamiento hacia atrás, comenzamos estableciendo $a = \text{True}$

Para $a = \text{True}$, observamos que puede derivarse de 2: $a \rightarrow b$. Dado que en la base de conocimiento se encuentran

Para $b = \text{True}$, utilizamos la regla 4: $b \rightarrow c$. Como ya sabemos que $c = \text{True}$ (R6), se concluye que $a = \text{True}$.

Al tener $a = \text{True}$ y $b = \text{True}$, la regla 1 nos permite concluir que $c = \text{True}$.

En cuanto a la propiedad necesaria del algoritmo para asegurar que esta derivación sea posible

3.3 La CNF de la base de conocimiento se presenta en las cláusulas 1 a 7. Suponiendo $\neg a$ y aplicando

Resolvemos: CNF de la base de conocimiento

A partir de cada regla $a \rightarrow b$ y de los hechos: 1: $a \rightarrow b$ 1: $\neg a \vee b$ 2: $b \rightarrow c$ 2: $\neg b \vee c$

3: $c \rightarrow a$ 3: $\neg c \vee a$ 4: $a \rightarrow b$ 4: $\neg a \vee b$ 5: b 5: b 6: c 6: c 7: $a \rightarrow b$ 7: $\neg a \vee b$ CNF(KB) = { 1,..., 7 }

Demostración por contradicción (resolución)

Queremos probar a . Por refutación añadimos $\neg a$: 8: $\neg a$ (negación de la meta) Resoluciones: De

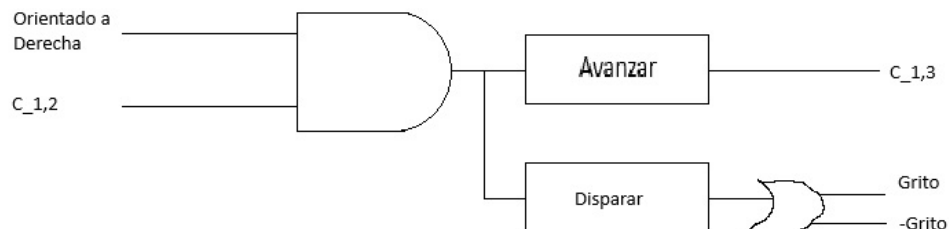
2: $\neg b \vee c$ con 5: 9: c (resolviendo en b) De 9: c con 6: 10: a (resolviendo en c) De

4: $\neg c \vee a$ con 6: 11: a (resolviendo en c) De 1: $\neg a \vee b$ con 10: 12: b (resolviendo en a) De

12: b con 11: 13: a (resolviendo en b) De 13: a con 8: $\neg a$ (cláusula vacía)

Como se deriva contradicción, KB $\models a$. Por lo tanto, $a = \text{True}$

4. Diseñe con lógica proposicional basada en circuitos las proposiciones *OrientadoDerecha* y *Agente ubicado en la casilla [1,2]* para el mundo de wumpus de 4x4. Dibuje el circuito correspondiente.



5. El nonograma es un juego en el cual se posee un tablero en blanco y cada fila y columna presenta información sobre la longitud de un bloque en dicha fila/columna. Además, la leyenda puede indicar más de un número, indicando esto que existen varios bloques de las longitudes mostradas por la leyenda y en el mismo orden, separados por al menos un espacio vacío.

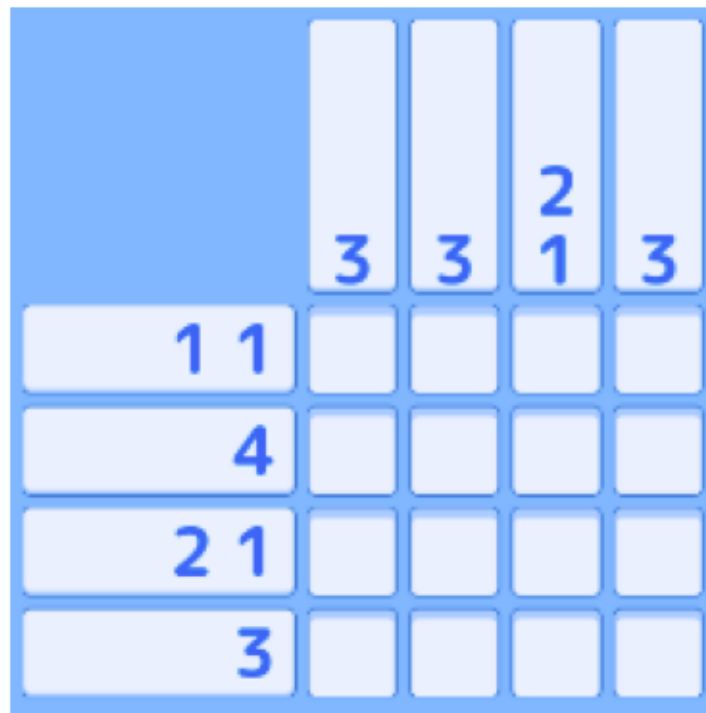
Resuelva el nonograma de la imagen de abajo escribiendo en primer lugar cada regla que puede incorporarse a la base de conocimientos inicial e incorporando cada inferencia que realice.

```
[1]: import requests
from PIL import Image
from io import BytesIO
import matplotlib.pyplot as plt

# URL directa de Google Drive
url = "https://drive.google.com/uc?
      ↪export=view&id=1SKiXvrI_TX-U4sbw60TYSRmaNYyFixmI"

# Descargar la imagen
response = requests.get(url)
img = Image.open(BytesIO(response.content))

# Mostrar la imagen
plt.imshow(img)
plt.axis('off') # Ocultar ejes
plt.show()
```



Interpretando las pistas: El tablero es de 4x4 Las columnas de izquierda a derecha muestran: Col 1: [3]; Col 2: [3]; Col 3: [2,1]; Col 4: [3]; Fil 1: [1,1]; Fil 2: [4]; Fil 3[2,1]; Fil 4: [3].

Reglas iniciales:Cada pista se transforma en reglas de la base de conocimiento (ejemplos):

R1:“Col 1 = [3]” → hay 3 negras consecutivas en la columna 1. → (C11 C21 C31)v(C21 C31 C41)

R2: “Col 2 = [3]” → hay 3 negras consecutivas en la columna 2 → (C12 C22 C32)v(C22 C32 C42)

R3:“Col 3 = [2,1]” → un bloque de 2 negras y luego un bloque de 1 negra. → (C13 C23 C43)

R4: “Col 4 = [3]” → hay 3 negras consecutivas en la columna 4. → (C14 C24 C34)v(C24 C34 C44)

R5: “Fila 1 = [1,1]” → debe haber dos bloques de 1 negro separados al menos por una blanca. → (C11 C13)v(C12 C14)

R6: “Fila 2 = [4]” → todas las casillas de la fila 2 son negras. → C21 C22 C23 C24

R7: “Fila 3 = [2,1]” → un bloque de 2 negras, luego una blanca, luego un bloque de 1 negra. → C31 C32 C34

R8: “Fila 4 = [3]” → hay exactamente 3 negras consecutivas en la fila 4. → (C41 C42 C43)v(C42 C43 C44)

Inferencias paso a paso

I1: de R6 (Fila 2 = [4]) → las 4 casillas de la fila 2 son negras:(2,1), (2,2), (2,3), (2,4) = negras. C21 C22 C23 C24

I2: de R7 (Fila 3 = [2,1]) → las primeras 2 casillas de la fila 3 son negras, la tercera es blanca y la cuarta es negra:(3,1), (3,2)y (3,4) = negras; (3,3) = blanca. C31 C32 C34

I3: de R3 (Col 3 = [2,1]) → por I2, la casilla (3,3) es blanca, y por I1, la casilla (1,3) es negra. Por lo que las otras 2 de la columna 3 son negras:(1,3), (2,3) y (4,3) = negras; (3,3) = blanca.

I4: de R5, (Fila 1 = [1,1]) → Como ya se conoce que el color de la casilla (1,3) es negra, y como deben haber dos bloques de 1 negro separados al menos por una blanca, se sabe que tanto (1,2) como (1,4) son blancas, llegando a la conclusión de que la casilla (1,1) es negra:(1,1) y (1,3) = negras; (1,2) y (1,4) = blancas.

I5: por R1 (Col 1 = [3]) → hay 3 negras consecutivas en la columna 1. Y gracias a I1, I2, e I4, se sabe que las casillas negras son la (1,1), (2,1) y (3,1); por lo que la casilla (4,1) debe ser blanca.

I6: por R8 (Fila 4 = [3]) → hay exactamente 3 negras consecutivas en la fila 4. Por I3 se sabe que la casilla (4,3) es negra y por I5 sabemos que la casilla (4,1) es blanca. Por esto mismo, siguiendo la regla 8, obtenemos que las dos casillas restantes deben ser negras.

Retomando, escribiendolo de manera de manera lógica: I1: C21 C22 C23 C24

I2: C31 C32 C34

I3: C13 C23 C43

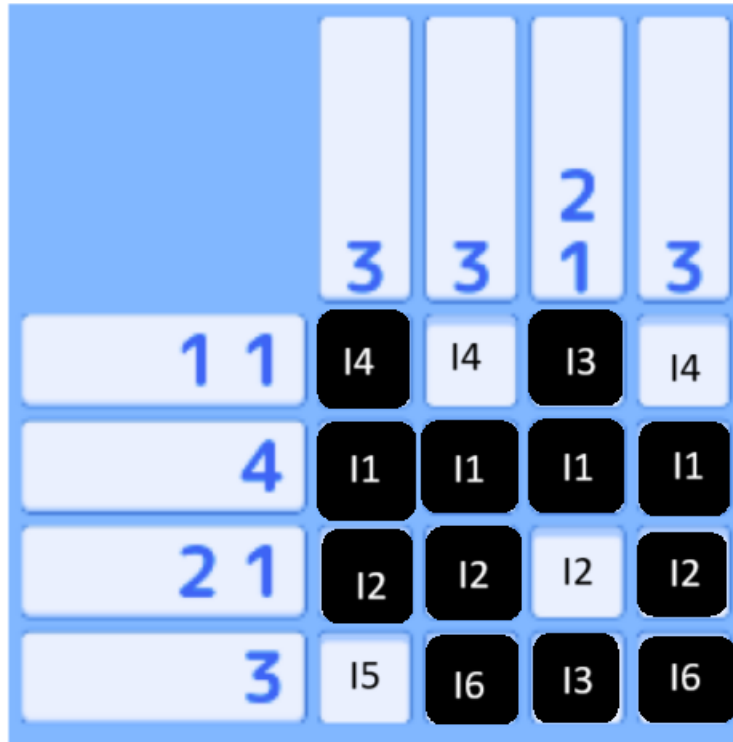
I4: Dado: - R5: (C11 C13) (C12 C14) - C13 = - C12 = - C14 = % Entonces: - Segunda disyunción: (C12 C14) = - Fórmula queda: (C11) C11 C11 C11 = T

I5: Dado: - R1: (C11 C21 C31) (C21 C31 C41) - C11, C21, C31 =

Entonces: - Primera disyunción ya satisface la regla - No puede haber más de 3 negras consecutivas
 $\neg C41 = \quad C41 = \text{Falso}$

I6: Dado: - R8: (C41 C42 C43) (C42 C43 C44) - C43 = - C41 =

Entonces: - Primera disyunción = - Segunda disyunción = (C42 C44) C42 = C44 =



1.2 Ejercicios de Implementación

- Implementar un motor de inferencia con encadenamiento hacia adelante. Pruébalo con las proposiciones del ejercicio 3.

```
[2]: from dataclasses import dataclass, field
from typing import Tuple, Set, List

'''
Este script implementa un motor de inferencia proposicional (reglas de Horn)
    usando
encadenamiento hacia adelante (Forward Chainer):

¿Cómo funciona?

1. Se parte de una base de conocimientos con hechos iniciales y reglas del
    tipo
    "si se cumplen ciertas condiciones, entonces se concluye algo".
```

2. El motor revisa todas las reglas: si todas las premisas de una regla ya están en los hechos conocidos y la conclusión aún no, agrega la conclusión como nuevo hecho.
3. Este proceso se repite: cada vez que se deduce un nuevo hecho, se vuelve a revisar si permite aplicar más reglas.
4. El ciclo termina cuando no se pueden deducir más hechos nuevos.

El código se inicializa con una base de conocimientos (kb).
Mantiene un conjunto de hechos derivados (self.derived), que comienza con los hechos iniciales.

El método infer aplica las reglas repetidamente: si todas las premisas de una regla están en los hechos derivados y la conclusión aún no está, añade la conclusión a los hechos derivados.

El proceso se repite hasta que no se puedan derivar más hechos nuevos.

Cada vez que se aplica una regla, se imprime qué regla se usó y qué hecho se dedujo.

'''

```
# Clase que representa una regla logica de Horn (premisas -> conclusión)
# Premises: conjunto de símbolos atómicos que deben ser verdaderos
# Head: símbolo atómico que se infiere si todas las premisas son verdaderas
# Name: etiqueta opcional para referenciar la regla en las explicaciones
```

```
@dataclass(frozen=True)
class Rule:
    premises: Tuple[str, ...]
    head: str
    name: str = ""
```

```
#Clase que representa la base de conocimientos
```

```
@dataclass
class KnowledgeBase:

    # Facts: conjunto de hechos conocidos inicialmente
    # Rules: conjunto de reglas lógicas de Horn

    facts: Set[str] = field(default_factory=set)
    rules: List[Rule] = field(default_factory=list)
```

```

#Metodo que agrega un hecho a la base de conocimientos

def add_fact(self, f: str) -> None:
    self.facts.add(f)

#Metodo que agrega una regla a la base de conocimientos

def add_rule(self, rule: Rule) -> None:
    self.rules.append(rule)

#Clase que implementa el motor de inferencia por encadenamiento hacia adelante

class ForwardChainer:

    #Inicializa el motor con una base de conocimientos

    def __init__(self, kb: KnowledgeBase):
        self.kb = kb
        self.derived = set(kb.facts)

    #Metodo que realiza la inferencia aplicando las reglas repetidamente

    def infer(self, verbose: bool = True):
        new_inferred = True
        while new_inferred:
            new_inferred = False
            for rule in self.kb.rules:
                if rule.head not in self.derived and all(p in self.derived for
↪p in rule.premises):
                    self.derived.add(rule.head)
                    new_inferred = True
                    if verbose:
                        print(f"Aplicando {rule.name or rule}: {' '.join(rule.
↪premises)} {rule.head}")
                    return self.derived

# Construcción de la base de conocimiento del ejercicio 3

kb = KnowledgeBase()
kb.add_fact("d")
kb.add_fact("e")
kb.add_rule(Rule(premises=("b", "c"), head="a", name="R1"))
kb.add_rule(Rule(premises=("d", "e"), head="b", name="R2"))
kb.add_rule(Rule(premises=("g", "e"), head="b", name="R3"))
kb.add_rule(Rule(premises=("e",), head="c", name="R4"))
kb.add_rule(Rule(premises=("a", "g"), head="f", name="R7"))

```

```
#Llamado al motor de inferencia que muestra el proceso y los pasos intermedios
```

```
fc = ForwardChainer(kb)
print("Hechos iniciales:", kb.facts)
result = fc.infer(verbose=True)
print("Hechos inferidos:", result)
```

Hechos iniciales: {'d', 'e'}

Aplicando R2: d e b

Aplicando R4: e c

Aplicando R1: b c a

Hechos inferidos: {'d', 'b', 'e', 'a', 'c'}

7. Implementar un motor de inferencia con encadenamiento hacia atrás. Pruébalo con las proposiciones del ejercicio 3.

```
[ ]: from __future__ import annotations
from dataclasses import dataclass, field
from typing import List, Set, Dict, Tuple, Optional

"""
Backward chaining inference engine (encadenamiento hacia atrás) - MUY comentado
↪ en español.

Este script implementa un motor de inferencia proposicional de Horn (reglas del
↪ tipo
premisas_1 premisa_2 ... premisa_n -> conclusión) usando encadenamiento
↪ hacia atrás.

Cómo funciona el encadenamiento hacia atrás
-----
Dado un objetivo (por ejemplo, 'a'), el motor intenta demostrarlo "yendo hacia
↪ atrás":
1) Si el objetivo ya es un hecho conocido, se da por demostrado.
2) Si no, busca reglas cuya conclusión sea el objetivo.
3) Para cada regla candidata, intenta demostrar *recursivamente* cada una de
↪ sus premisas.
4) Si todas las premisas de alguna regla se demuestran, entonces el objetivo
↪ queda demostrado.
5) Se evita el bucle con dos mecanismos: (a) una *pila de metas* (goals_stack)
↪ para detectar ciclos,
y (b) *memorización* de metas que ya fueron probadas como verdaderas o
↪ imposibles.

El algoritmo es correcto para bases de conocimiento con reglas de Horn, que es
↪ justamente el
```


formato del ejercicio 3 del TP (R1..R7).

Además del veredicto True/False, registramos un *árbol de prueba* para explicar
→ QUÉ reglas

permitieron demostrar cada meta.

"""

```
# -----  
# Representación del Conocimiento  
# -----
```

```
@dataclass(frozen=True)
```

```
class Rule:
```

```
    """
```

```
    Regla de Horn: premisas -> conclusión.
```

- premises: conjunto de símbolos atómicos que deben ser verdaderos
- head: símbolo atómico que se infiere si todas las premisas son verdaderas
- name: etiqueta opcional para referenciar la regla en las explicaciones (p.

→ej., 'R1')

```
    """
```

```
    premises: Tuple[str, ...]
```

```
    head: str
```

```
    name: str = ""
```

```
@dataclass
```

```
class KnowledgeBase:
```

```
    """
```

```
    Base de conocimiento con:
```

- facts: hechos atómicos ya conocidos como verdaderos (p.ej., {'d', 'e'})
- rules: lista de reglas de Horn (Rule)

También construimos un índice de acceso rápido para recuperar "reglas que
→concluyen X".

```
    """
```

```
    facts: Set[str] = field(default_factory=set)
```

```
    rules: List[Rule] = field(default_factory=list)
```

```
    _by_head: Dict[str, List[Rule]] = field(init=False, default_factory=dict)
```

```
    def __post_init__(self):
```

```
        self._rebuild_index()
```

```
    def _rebuild_index(self) -> None:
```

"""Índice 'conclusión -> [reglas]' para acelerar el buscado de reglas
→candidatas."""

```
        self._by_head.clear()
```

```

        for r in self.rules:
            self._by_head.setdefault(r.head, []).append(r)

    def add_fact(self, f: str) -> None:
        self.facts.add(f)

    def add_rule(self, rule: Rule) -> None:
        self.rules.append(rule)
        self._by_head.setdefault(rule.head, []).append(rule)

    def rules_for(self, head: str) -> List[Rule]:
        """Devuelve todas las reglas cuya 'head' coincide con 'head'."""
        return self._by_head.get(head, [])

# -----
# Estructuras para la explicación (árbol de prueba)
# -----

@dataclass
class ProofNode:
    """
    Nodo del árbol de prueba:
    - goal: meta demostrada en este nodo (ej., 'a')
    - rule: regla que permitió concluir esa meta (o None si fue un hecho base)
    - children: subpruebas para cada premisa de la regla
    """
    goal: str
    rule: Optional[Rule] = None
    children: List["ProofNode"] = field(default_factory=list)

    def pretty(self, level: int = 0) -> str:
        """Devuelve una representación legible del árbol de prueba."""
        indent = " " * level
        if self.rule is None:
            return f"{indent}- {self.goal} (hecho)"
        title = f"{indent}- {self.goal} (por {self.rule.name or 'regla'})"
        subs = "\n".join(child.pretty(level + 1) for child in self.children)
        return f"{title}\n{subs}" if subs else title

# -----
# Motor de encadenamiento hacia atrás
# -----

class BackwardChainer:
    """

```

Motor básico de encadenamiento hacia atrás para reglas de Horn.

Soporta:

- *Memoización de éxitos y fracasos para eficiencia.*
- *Detección de ciclos con una pila de metas.*
- *Generación de árbol de prueba inteligible.*

"""

```
def __init__(self, kb: KnowledgeBase):
    self.kb = kb
    # Cache de metas ya resueltas: símbolo -> (éxito, ProofNode opcional)
    self._cache_success: Dict[str, ProofNode] = {}
    self._cache_failure: Set[str] = set()
```

```
def prove(self, goal: str) -> Tuple[bool, Optional[ProofNode]]:
```

"""

Intenta demostrar 'goal'. Devuelve (True, prueba) si lo logra, o
↪ (False, None) si no.

"""

```
    # Usamos una pila para detectar ciclos (ej.: a depende de b, b de a).
    stack: List[str] = []
    success, node = self._prove_recursive(goal, stack)
    return success, node
```

```
def _prove_recursive(self, goal: str, stack: List[str]) -> Tuple[bool,
↪ Optional[ProofNode]]:
```

1) Ya lo conocés como hecho

```
if goal in self.kb.facts:
    return True, ProofNode(goal=goal, rule=None, children=[])
```

2) Evitar bucles: si la meta ya está en la pila, hay dependencia
↪ circular

```
if goal in stack:
    return False, None
```

3) Reutilizar resultados previos si existen

```
if goal in self._cache_success:
    return True, self._cache_success[goal]
if goal in self._cache_failure:
    return False, None
```

4) Buscar reglas que concluyan 'goal'

```
rules = self.kb.rules_for(goal)
if not rules:
    # No hay manera de probar esta meta con reglas: fracaso
    self._cache_failure.add(goal)
    return False, None
```

```

# 5) Intentar cada regla candidata en orden (DFS)
stack.append(goal) # Push para detección de ciclo en sub-ramas
try:
    for rule in rules:
        # Intentar demostrar todas las premisas de esta regla
        children: List[ProofNode] = []
        all_premises_ok = True
        for prem in rule.premises:
            ok, child = self._prove_recursive(prem, stack)
            if not ok:
                all_premises_ok = False
                break
            assert child is not None
            children.append(child)

        if all_premises_ok:
            # ¡Éxito! Construimos el nodo de prueba y lo memorizamos
            node = ProofNode(goal=goal, rule=rule, children=children)
            self._cache_success[goal] = node
            # Nota: no añadimos goal a los hechos para mantener la
            ↪ explicación separada de los datos
            return True, node

        # Ninguna regla logró demostrar el objetivo
        self._cache_failure.add(goal)
        return False, None
finally:
    stack.pop() # Pop al volver de la rama actual

# -----
# Utilidades de ayuda
# -----

def build_kb_from_ex3() -> KnowledgeBase:
    """
    Construye la base de conocimiento EXACTA del Ejercicio 3 del TP:

    R1: b   c → a
    R2: d   e → b
    R3: g   e → b
    R4: e → c
    R5: d
    R6: e
    R7: a   g → f
    """
    kb = KnowledgeBase()

```

```

# Hechos (R5 y R6 son reglas unitarias, las tomamos como hechos)
kb.add_fact("d")
kb.add_fact("e")

# Reglas no unitarias
kb.add_rule(Rule(premises=("b", "c"), head="a", name="R1"))
kb.add_rule(Rule(premises=("d", "e"), head="b", name="R2"))
kb.add_rule(Rule(premises=("g", "e"), head="b", name="R3"))
kb.add_rule(Rule(premises=("e",), head="c", name="R4")) # también podría
→ modelarse como hecho si e es hecho
kb.add_rule(Rule(premises=("a", "g"), head="f", name="R7"))
return kb

def demo():
    """
    Demostración breve:
    - Intenta probar 'a' (debería ser True).
    - Intenta probar 'f' (debería fallar porque 'g' no se puede probar).
    Imprime los árboles de prueba cuando existan.
    """
    kb = build_kb_from_ex3()
    engine = BackwardChainer(kb)

    print("=== Prueba de 'a' ===")
    ok_a, proof_a = engine.prove("a")
    print("Resultado:", ok_a)
    if ok_a and proof_a:
        print(proof_a.pretty())

    print("\n=== Prueba de 'f' ===")
    ok_f, proof_f = engine.prove("f")
    print("Resultado:", ok_f)
    if ok_f and proof_f:
        print(proof_f.pretty())
    else:
        print("No se pudo demostrar 'f' porque falta 'g'.")

if __name__ == "__main__":
    demo()

```

```
=== Prueba de 'a' ===
```

```
Resultado: True
```

- a (por R1)
 - b (por R2)
 - d (hecho)
 - e (hecho)
 - c (por R4)

- e (hecho)

=== Prueba de 'f' ===

Resultado: False

No se pudo demostrar 'f' porque falta 'g'.

8. Implementar un motor de inferencia por contradicción que detecte si el conjunto de proposiciones del ejercicio 3 es inconsistente.

```
[1]: """
Motor de inferencia por contradicción (resolución proposicional).

Idea principal
-----
Un conjunto de proposiciones (en CNF) es inconsistente si, aplicando
la regla de resolución entre sus cláusulas, se puede derivar la
cláusula vacía (). La cláusula vacía representa una contradicción.

Representación
-----
- Cada literal es una tupla (símbolo:str, es_negada:bool), por ejemplo:
   $\hookrightarrow ('a', False) = a, ('a', True) = \neg a.$ 
- Cada cláusula es un `frozenset` de literales (disyunción de literales).
- Una CNF es un `set` de cláusulas (conjunción de cláusulas).

Regla de resolución
-----
Dadas dos cláusulas C1 y C2, y un símbolo p, si en C1 está p y en C2 está  $\neg p$ 
(entonces "chocan" en p), el resolvente es:
    resolvente = (C1 - {p}) (C2 - { $\neg p$ })
Si en algún momento obtenemos el conjunto vacío, se detectó una contradicción.

Este archivo incluye además la traducción a CNF de las reglas del Ejercicio 3:
R1: b    c  $\rightarrow$  a       $\rightarrow (\neg b \quad \neg c \quad a)$ 
R2: d    e  $\rightarrow$  b       $\rightarrow (\neg d \quad \neg e \quad b)$ 
R3: g    e  $\rightarrow$  b       $\rightarrow (\neg g \quad \neg e \quad b)$ 
R4: e  $\rightarrow$  c               $\rightarrow (\neg e \quad c)$ 
R5: d                                 $\rightarrow (d)$ 
R6: e                                 $\rightarrow (e)$ 
R7: a    g  $\rightarrow$  f       $\rightarrow (\neg a \quad \neg g \quad f)$ 

Como solo queremos chequear inconsistencia del conjunto, NO añadimos
la negación de ninguna consulta. Si la CNF por sí sola deriva , es
 $\hookrightarrow$  inconsistente.
"""
from __future__ import annotations
from typing import Set, FrozenSet, Tuple, Iterable
```

```

Literal = Tuple[str, bool]    # (symbol, is_negated) ; is_negated=True
    ↪ representa ¬symbol
Clause  = FrozenSet[Literal] # disyunción de literales
CNF     = Set[Clause]        # conjunción de cláusulas

# -----
# Utilidades sobre literales
# -----

def neg(lit: Literal) -> Literal:
    """Negación de un literal: (p, False) -> (p, True) y viceversa."""
    return (lit[0], not lit[1])

def has_complement(c1: Clause, c2: Clause) -> Iterable[Tuple[Literal, Literal]]:
    """
    Devuelve pares (l, ¬l) tales que l ∈ c1 y ¬l ∈ c2.
    Sirve para saber en qué literales se pueden resolver C1 y C2.
    """
    s2 = set(c2)
    for l in c1:
        comp = neg(l)
        if comp in s2:
            yield (l, comp)

def clause_to_str(c: Clause) -> str:
    """Imprime una cláusula de forma legible, p. ej. (¬b ¬c a) o () si es
    ↪vacía."""
    if not c:
        return " " # cláusula vacía (contradicción)
    lits = []
    for sym, is_neg in sorted(c, key=lambda x: (x[0], x[1])):
        lits.append(("¬" if is_neg else "") + sym)
    return "(" + " ".join(lits) + ")"

def cnf_to_str(F: CNF) -> str:
    """Imprime una CNF (conjunción de cláusulas)."""
    if not F:
        return "VERDAD (CNF vacía)"
    return " ".join(sorted([clause_to_str(c) for c in F]))

# -----
# Resolución proposicional
# -----

def resolve(c1: Clause, c2: Clause) -> Set[Clause]:

```

```

    """
    Aplica resolución entre C1 y C2. Devuelve el conjunto de resolventes
    ↪ posibles
    (podría haber más de uno si hay varios literales complementarios).
    Si se obtiene la cláusula vacía, se incluirá como `frozenset()` en el
    ↪ resultado.
    """
    resolvents: Set[Clause] = set()
    for l, nl in has_complement(c1, c2):
        new_clause = frozenset((c1 - {l}) | (c2 - {nl}))
        # Simplificaciones básicas: quitar literales repetidos y detectar
    ↪ tautologías p ¬p
        if any((sym, True) in new_clause and (sym, False) in new_clause for
    ↪ sym, _ in new_clause):
            # Es una tautología -> no aporta nada, se puede ignorar
            continue
        resolvents.add(new_clause)
    return resolvents

def resolution_refutation(F: CNF, verbose: bool = True) -> bool:
    """
    Intenta derivar la cláusula vacía por resolución a partir de F.
    Devuelve True si se encuentra (inconsistencia), False si no (hasta
    ↪ saturar).

    Estrategia: algoritmo de resolución por saturación clásico (pairwise).
    """
    # Trabajamos con una copia para no modificar el original
    clauses: Set[Clause] = set(F)
    new: Set[Clause] = set()

    if verbose:
        print("CNF inicial:")
        for c in clauses:
            print(" ", clause_to_str(c))

    # Bucle de saturación: intentar todas las resoluciones posibles
    while True:
        pairs = []
        cls_list = list(clauses)
        n = len(cls_list)
        for i in range(n):
            for j in range(i + 1, n):
                pairs.append((cls_list[i], cls_list[j]))

        generated_any = False
        for (c1, c2) in pairs:

```



```

        resolvents = resolve(c1, c2)
        for r in resolvents:
            if not r:
                if verbose:
                    print("\nSe derivó la cláusula vacía por resolución de:
↪")

                    print("    ", clause_to_str(c1))
                    print("    ", clause_to_str(c2))
                    print("=>    (INCONSISTENTE)")
                    return True # inconsistente
            if r not in clauses and r not in new:
                new.add(r)
                generated_any = True
                if verbose:
                    print("Nuevo resolvente:", clause_to_str(r))

        if not generated_any:
            # No se generaron más cláusulas -> saturado sin llegar a
            if verbose:
                print("\nSaturado sin contradicción. No se pudo derivar .")
            return False
        clauses |= new
        new.clear()

# -----
# Construir la CNF del Ejercicio 3
# -----

def lit(p: str, negado: bool = False) -> Literal:
    return (p, negado)

def cnf_from_ex3() -> CNF:
    """
    Traducción directa de las reglas y hechos del Ejercicio 3 a CNF:
    ( $\neg b \neg c a$ ), ( $\neg d \neg e b$ ), ( $\neg g \neg e b$ ), ( $\neg e c$ ), ( $d$ ), ( $e$ ), ( $\neg a \neg g f$ )
    """
    F: CNF = set()
    # R1:  $b \ c \rightarrow a \implies (\neg b \neg c a)$ 
    F.add(frozenset({lit('b', True), lit('c', True), lit('a', False)}))
    # R2:  $d \ e \rightarrow b \implies (\neg d \neg e b)$ 
    F.add(frozenset({lit('d', True), lit('e', True), lit('b', False)}))
    # R3:  $g \ e \rightarrow b \implies (\neg g \neg e b)$ 
    F.add(frozenset({lit('g', True), lit('e', True), lit('b', False)}))
    # R4:  $e \rightarrow c \implies (\neg e c)$ 
    F.add(frozenset({lit('e', True), lit('c', False)}))
    # R5:  $d \implies (d)$ 

```

```

F.add(frozenset({lit('d', False)}))
# R6: e ==> (e)
F.add(frozenset({lit('e', False)}))
# R7: a g → f ==> (¬a ¬g f)
F.add(frozenset({lit('a', True), lit('g', True), lit('f', False)}))
return F

# -----
# Demostración con el Ejercicio 3
# -----

def demo():
    print("=== Chequeo de inconsistencia por resolución (Ejercicio 3) ===\n")
    F = cnf_from_ex3()
    inconsistente = resolution_refutation(F, verbose=True)
    print("\n¿La base es inconsistente?", inconsistente)
    if not inconsistente:
        print("Interpretación: las reglas + hechos NO implican contradicción_
↪lógica.")

if __name__ == "__main__":
    demo()

```

=== Chequeo de inconsistencia por resolución (Ejercicio 3) ===

CNF inicial:

```

(a ¬b ¬c)
(d)
(b ¬e ¬g)
(b ¬d ¬e)
(c ¬e)
(¬a f ¬g)
(e)

```

```

Nuevo resolvente: (a ¬c ¬e ¬g)
Nuevo resolvente: (a ¬c ¬d ¬e)
Nuevo resolvente: (a ¬b ¬e)
Nuevo resolvente: (¬b ¬c f ¬g)
Nuevo resolvente: (b ¬e)
Nuevo resolvente: (b ¬g)
Nuevo resolvente: (b ¬d)
Nuevo resolvente: (c)
Nuevo resolvente: (a ¬e)
Nuevo resolvente: (¬c ¬e f ¬g)
Nuevo resolvente: (a ¬c ¬e)
Nuevo resolvente: (b)
Nuevo resolvente: (a ¬e ¬g)
Nuevo resolvente: (¬c f ¬g)

```

Nuevo resolvente: (a $\neg$ c $\neg$ g)
 Nuevo resolvente: ($\neg$ b $\neg$ e f $\neg$ g)
 Nuevo resolvente: (a $\neg$ d $\neg$ e)
 Nuevo resolvente: (a $\neg$ b)
 Nuevo resolvente: ($\neg$ b f $\neg$ g)
 Nuevo resolvente: ($\neg$ c $\neg$ d $\neg$ e f $\neg$ g)
 Nuevo resolvente: ($\neg$ c $\neg$ d f $\neg$ g)
 Nuevo resolvente: (a $\neg$ c $\neg$ d)
 Nuevo resolvente: (a $\neg$ g)
 Nuevo resolvente: (f $\neg$ g)
 Nuevo resolvente: ($\neg$ e f $\neg$ g)
 Nuevo resolvente: (a $\neg$ d)
 Nuevo resolvente: ($\neg$ d $\neg$ e f $\neg$ g)
 Nuevo resolvente: ($\neg$ d f $\neg$ g)
 Nuevo resolvente: (a $\neg$ c)
 Nuevo resolvente: (a)

Saturado sin contradicción. No se pudo derivar .

¿La base es inconsistente? False

Interpretación: las reglas + hechos NO implican contradicción lógica.

2 Bibliografía

Russell, S. & Norvig, P. (2004) *Inteligencia Artificial: Un Enfoque Moderno*. Pearson Educación S.A. (2a Ed.) Madrid, España

Poole, D. & Mackworth, A. (2023) *Artificial Intelligence: Foundations of Computational Agents*. Cambridge University Press (3a Ed.) Vancouver, Canada